Stored XSS into HTML context with nothing encoded

Topic: Cross-Site Scripting (XSS) | Difficulty: Apprentice

Description

The blog comment section stores user-supplied HTML without sanitization or encoding. Injected JavaScript is persisted in the database and executes in every user's browser when they view the affected page, making this a persistent high-impact XSS vulnerability.

Steps to Exploit

1. Navigate to a blog post that allows user comments.
2. In the comment body field enter: <script>alert(document.domain)</script>
3. Submit the comment form.
4. Navigate to the blog post as any user.
5. Observe the alert dialog fires automatically on page load.

Proof of Concept

Payload (comment field):

```
<script>alert(document.domain)</script>
```

Effect:

```
Script persists in the database and fires for every visitor to the page.
```

Impact

- Persistent script execution for all users viewing the affected page.
- Mass session token theft from all visitors.
- Higher severity than reflected XSS due to persistent and widespread impact.
- Automated propagation of malicious content to all page visitors.

Mitigation / Remediation

1. Apply HTML entity encoding on all stored user content before rendering.
2. Implement a Content Security Policy (CSP) to restrict inline script execution.
3. Validate and sanitize input at storage time using a well-tested HTML sanitizer.
4. Mark session cookies as HttpOnly and Secure.

CVSS Score

CVSS v3.1: 6.1 (Medium)

Vector: CVSS:3.1/AV:N/AC:L/PR:N/UI:R/S:C/C:L/I:L/A:N

CVSS Justification

- Attack Vector: Network - injected via web form.
- Attack Complexity: Low.
- Privileges Required: None - comment posting is open.

- User Interaction: Required - victims must visit the page (passive trigger).
- Scope: Changed - code runs in victims' browser contexts.
- Confidentiality / Integrity: Low - session theft and page manipulation.

XSS Stored

Proof of Concept — Exploitation Evidence

Payload / exploit steps are documented in the **Proof of Concept** section of this writeup (above). Screenshot evidence of successful exploitation is shown below.

Lab Solved Confirmation

PortSwigger lab status changed to “Solved” after exploitation.